README | Webots PY Cycl

Proyecto wbtlab11_Py_Cycl: persecución cíclica orbital de cuatro robots Pololu 3Pi, sin líder fijo.

Organización del control

Los cuatro robots ejecutan la misma ley de control. Cada uno consulta al siguiente y apunta a una posición adelantada sobre un círculo común. NEXT_DEF define el anillo:

Archivo en controllers/	NEXT_DEF	Relación
robot1/robot1.py	ROBOT2	Agente 1 persigue al 2.
robot2/robot2.py	ROBOT3	Agente 2 persigue al 3.
robot3/robot3.py	ROBOT4	Agente 3 persigue al 4.
robot4/robot4.py	ROBOT1	Agente 4 cierra el ciclo hacia el 1.

Ningún robot genera una trayectoria temporal para los demás. Todos crean un objeto Supervisor, leen su propio GPS y Compass, consultan la posición del siguiente agente y accionan sus ruedas.

Estructura del proyecto

Ruta	Contenido
worlds/Trayectory_4agents.wbt	Único mundo, formato R2025a; asigna robot1, robot2, robot3 y robot4.
controllers/robot1/ ... robot4/	Controladores Python de la persecución cíclica.
Modelos_3D/Pololu/ y protos/	Mallas STL y modelo Pololu3pi.proto.
Otros controladores y robotat/	Auxiliares MATLAB heredados, no activos en el mundo.

Requisitos y ejecución

1. Extraer el ZIP completo y conservar las carpetas. Usar Webots con Python configurado. Los scripts activos importan math y controller, el módulo proporcionado por Webots; no requieren NumPy ni MATLAB.

2. Sustituir en el mundo las rutas absolutas de BumperV2.stl por esta ruta relativa al archivo incluido:

```
../Modelos_3D/Pololu/BumperV2.stl
```

3. Abrir worlds/Trayectory_4agents.wbt y ejecutar la simulación desde Webots. Los scripts se inician como controladores de los robots, no como programas independientes de terminal.

4. Conservar los DEF ROBOT1, ROBOT2, ROBOT3 y ROBOT4, las asignaciones robot1...robot4 y supervisor TRUE en los cuatro nodos. NEXT_DEF busca el identificador DEF, no el campo name.

Cada robot necesita los dispositivos motor_1, motor_2, gps y compass. El mundo define los robots directamente; no instancia Pololu3pi.proto.

Objetivos y ley de control

Parámetros comunes de los cuatro scripts

Parámetro	Valor configurado
TIME_STEP	64 ms
CENTER_X / CENTER_Y	0 / 0 mm
Ra	1000 mm: radio de referencia de 1 m.
N_AGENTS	4
PHI_OFFSET	1.5*math.pi/N_AGENTS = 3*pi/8 rad = 67.5 grados.
v0 / alpha	150 mm/s / 0.0001
kpO / kiO / kdO	6 / 0.0001 / 0.1
w_max	6 rad/s
v_max	400 rpm por rueda.

Punto objetivo de cada agente

La posición del siguiente robot se lee mediante getFromDef(NEXT_DEF) y su campo translation. Sus coordenadas y las del GPS propio se convierten de metros a milímetros. El objetivo se calcula en cada paso:

```
ang_next = math.atan2(ny - CENTER_Y, nx - CENTER_X)
ang_target = ang_next + PHI_OFFSET
xg = CENTER_X + Ra*math.cos(ang_target)
yg = CENTER_Y + Ra*math.sin(ang_target)
```

El objetivo se proyecta sobre el círculo de referencia usando el ángulo del siguiente robot, aunque este se aleje radialmente. PHI_OFFSET adelanta el objetivo en sentido angular positivo; no fija por sí solo la separación real entre agentes.

Errores y velocidad lineal

```
ex = xg - posx
ey = yg - posy
eP = math.hypot(ex, ey)
thetag = math.atan2(ey, ex)
eO = math.atan2(math.sin(thetag-theta),
                math.cos(thetag-theta))
```

Para eP > 1e-6 se usa la ley exponencial siguiente; para eP <= 1e-6, v = 0. No existe la zona de parada de 30 mm de la variante con líder.

```
v = v0*(1 - math.exp(-alpha*eP**2))
```

Velocidad angular PID

```
eO_D = eO - eO_1
EO = EO + eO
w = kpO*eO + kiO*EO + kdO*eO_D
eO_1 = eO
w = max(-w_max, min(w_max, w))
```

v se expresa en mm/s y w en rad/s. La suma integral no multiplica por dt y la diferencia derivativa no divide entre dt. El integrador sigue acumulándose cuando se satura w; no hay anti-windup.

Motores y configuración

Cinemática diferencial

Los controladores utilizan radio de rueda r = 32 mm y separación L = 82.4 mm. Las velocidades de rueda se calculan en rad/s y se convierten a rpm:

```
q_right = (2*v + w*L)/(2*r)
q_left  = (2*v - w*L)/(2*r)
rpm     = q*(60/(2*math.pi))
```

velsat limita cada rueda a +/-400 rpm. Después se convierte a rad/s y se invierte el signo según el montaje del robot en Webots:

```
left_motor.setVelocity(-rpm2rads(v_left))
right_motor.setVelocity(-rpm2rads(v_right))
```

La saturación de w ocurre antes de convertir a ruedas. Si además se satura una rueda, las velocidades efectivas del robot pueden diferir de las órdenes v y w. Webots calcula el movimiento físico; los scripts no integran RK4 ni escriben poses.

Qué modificar

Para cambiar el círculo, ajustar CENTER_X, CENTER_Y y Ra en los cuatro archivos. Mantener el mismo centro, radio y criterio de PHI_OFFSET si se busca una referencia compartida.

N_AGENTS solo interviene en el cálculo de PHI_OFFSET: cambiarlo no crea robots ni modifica los enlaces. Para ampliar el anillo hay que añadir nodos y controladores, asignar sus DEF y actualizar NEXT_DEF para cerrar el nuevo ciclo.

No se usan NV, tf, lookahead_samples ni d_sep. Los objetivos se actualizan a partir del siguiente robot y los ciclos continúan mientras robot.step(TIME_STEP) != -1. El radio de referencia no garantiza una órbita exacta ni una formación equiespaciada.

Detalles a revisar

Orientación: los cuatro controladores usan math.atan2(mag[1],mag[0]) - 90.0. atan2 devuelve radianes; si se pretende un ajuste de 90 grados, revisar math.pi/2 y comprobar los ejes y el signo en la escena.

Muestreo: GPS y Compass se habilitan a 64 ms. Verificar que TIME_STEP sea coherente con el basicTimeStep del mundo. Los controladores esperan un GPS válido antes de comenzar el ciclo principal.

Escena: no hay prevención de colisiones. Al cambiar el radio o el centro, revisar posiciones iniciales y espacio disponible. Los EXTERNPROTO dependen de recursos externos de Webots.

Auxiliares MATLAB heredados

central_controller.m, pololu3pi_controller.m, robot_receiver.m y la carpeta robotat/ permanecen en el ZIP. No se ejecutan en el mundo cíclico ni son versiones Python. Incluyen control centralizado, control individual, recepción de comandos y utilidades para Robotat.

Pololu3pi.proto conserva my_controller_EpuckPruebas, ausente del ZIP. Revisar esa asignación si se utiliza el PROTO en otro mundo. La simulación activa no abre conexiones con Robotat ni requiere robots físicos.

Resultados

La evolución del anillo se observa en Webots. Los controladores no generan gráficas, historiales ni archivos de resultados automáticamente.